

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

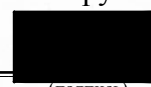
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Программирование систем аутентификации и авторизации

Выполнили:

Жук Анастасия Валерьевна,
студентка группы N3348



(подпись)

Проверил:

Ищенко Алексей Петрович

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ХОД РАБОТЫ.....	4
Техническое задание.....	4
Описание продукта, его функционал.....	7
Инструкция по применению.....	12
Код на языке программирования Python.....	16
Листинг 1 – Код консольного интерфейса приложения \client\console_client.py.....	16
Листинг 2 – Код основного файла FastAPI-приложения \server\main.py.....	32
Листинг 3 – Код определения SQLAlchemy-моделей \server\models.py.....	46
Листинг 4 – Код Pydantic-схем для валидации входных/выходных данных \server\schemas.py.....	48
Листинг 5 – Код модуля авторизации с проверкой прав доступа и зависимостями FastAPI \server\auth.py.....	50
Листинг 6 – Код альтернативных методов аутентификации \server\auth_methods.py.....	53
Листинг 7 – Код функций для работы с БД \server\crud.py.....	56
Листинг 8 – Код конфигурации подключения к PostgreSQL и создания сессий SQLAlchemy \server\database.py.....	60
Листинг 9 – Код модуля двухфакторной аутентификации \server\mfa.py.....	60
Листинг 10 – Код криптографических функций \server\security.py...	61
Листинг 11 – Код валидации email и телефонных номеров\server\validators.py.....	62
Примеры работы программы.....	64
ЗАКЛЮЧЕНИЕ.....	73

ВВЕДЕНИЕ

Цель работы — получить практический опыт разработки простой системы аутентификации и авторизации, изучить основные принципы проверки подлинности пользователей, управления сессиями и контроля доступа, а также повысить навыки программирования, работы с хэшированием, шифрованием и базами данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- создать базу данных для хранения информации о пользователях;
- реализовать модуль хэширования паролей с использованием безопасного алгоритма и солевой соли;
- разработать модуль аутентификации, проверяющий корректность введенных данных и генерирующий токены доступа;
- создать модуль авторизации для проверки прав доступа пользователей на основе ролей;
- реализовать модуль управления сессиями пользователей с использованием токенов;
- объединить разработанные модули в прототип системы с простым интерфейсом для взаимодействия с пользователем;
- выполнить дополнительные задания, направленные на расширение функциональности системы (смена пароля, многофакторная аутентификация, управление ролями).

ХОД РАБОТЫ

Техническое задание

Необходимые инструменты:

- Язык программирования – Python.
- Криптографическая библиотека для выбранного языка – hashlib.
- Текстовый редактор или IDE для написания кода – PyCharm.
- Инструменты для тестирования – ручное тестирование.
- Простая база данных с информацией о пользователях – PostgreSQL.

Задания:

1. Создание базы данных для хранения пользователей:
 - a. Создать базу данных с таблицей для хранения информации о пользователях (имя пользователя, пароль, роль).
 - b. Заполнить базу данных тестовыми данными.
2. Реализация модуля хэширования паролей:
 - a. Разработать модуль, который хэширует пароль пользователя с помощью безопасного алгоритма – argon2.
 - b. Использовать солевую соль для увеличения безопасности хэширования.
 - c. Сохранить хэшированный пароль в базе данных.
3. Реализация модуля аутентификации:
 - a. Разработать модуль, который проверяет корректность ввода имени пользователя и пароля и выдает токен аутентификации (JWT) в случае успешной аутентификации.
 - b. Сравнить хэшированный введенный пароль с хэшированным паролем из базы данных.
 - c. Сгенерировать токен аутентификации с использованием криптографически безопасного генератора.
4. Реализация модуля авторизации:

- a. Разработать модуль, который проверяет права доступа пользователя к ресурсам на основе токена аутентификации и информации в базе данных.
 - b. Реализовать разные методы проверки прав доступа (например, простая таблица прав, иерархическая модель прав).
- 5. Реализация модуля управления сессиями:
 - a. Разработать модуль, который управляет сессиями пользователей с использованием токенов аутентификации.
 - b. Реализовать функции создания новой сессии, обновления сессии и завершения сессии.
- 6. Создание прототипа простой системы аутентификации и авторизации:
 - a. Создать программу, которая объединяет модули хэширования, аутентификации, авторизации и управления сессиями.
 - b. Реализовать простой интерфейс для взаимодействия с пользователями (например, консольный интерфейс или веб-интерфейс).
- 7. Дополнительные задания:
 - a. Реализовать функцию смены пароля пользователем с проверкой старого пароля.
 - b. Разработать модуль для многофакторной аутентификации (например, с использованием SMS или email).
 - c. Реализовать функцию управления ролями пользователей (например, добавление, удаление, изменение ролей).
- 8. Рекомендации:
 - a. Изучить разные подходы к реализации систем аутентификации и авторизации – OAuth.
 - b. Использовать криптографические библиотеки вашего языка программирования для хэширования и шифрования.

- c. Создать тестовую базу данных с информацией о пользователях.
- d. Разработать простые правила для проверки прав доступа.

9. Важно:

- a. Помнить о безопасности данных в базе данных. Используйте шифрование и защиту от несанкционированного доступа.
- b. Создать сильную систему аутентификации, чтобы предотвратить несанкционированный доступ к системе.

Описание продукта, его функционал

Техническая архитектура системы

Разработанная система представляет собой комплексное решение для аутентификации и авторизации, реализованное в виде двух взаимосвязанных компонентов. Серверная часть построена на фреймворке FastAPI и предоставляет RESTful API для управления всеми операциями. Клиентская часть представляет собой консольное приложение на Python, которое служит удобным интерфейсом для взаимодействия с API сервера.

Архитектурно система организована по принципу клиент-сервер, что позволяет четко разделить ответственность: сервер отвечает за бизнес-логику, безопасность и хранение данных, а клиент – за представление информации и взаимодействие с пользователем. Такое разделение соответствует современным подходам к разработке веб-приложений и обеспечивает масштабируемость системы.

Структура базы данных и модели

В основе системы лежит реляционная база данных PostgreSQL, структура которой организована в виде шести взаимосвязанных таблиц:

1. Таблица пользователей (users) хранит основную информацию о каждом зарегистрированном пользователе. Каждая запись включает уникальный идентификатор, имя пользователя, электронную почту, телефонный номер, хэшированный пароль, ссылку на роль пользователя, настройки двухфакторной аутентификации, статус активности и временные метки создания. Пароли хранятся в безопасном хэшированном виде с использованием алгоритма Argon2.

2. Таблица ролей (roles) определяет права доступа в системе. Каждая роль имеет уникальное имя, описание, набор разрешений в формате JSON, ссылку на родительскую роль (для реализации иерархической модели) и идентификатор модели прав. В системе предустановлены три базовые роли: администратор (полный доступ),

модератор (ограниченное управление пользователями) и обычный пользователь (доступ только к своему профилю).

3. Таблица сессий (`sessions`) управляет активными пользовательскими сессиями. Для каждой сессии хранится уникальный токен сессии, `refresh`-токен, время истечения, информация о браузере пользователя и IP-адрес. Такая структура позволяет эффективно управлять сессиями и обеспечивать безопасность.

4. Таблица API-ключей (`api_keys`) предназначена для аутентификации через API-ключи. Ключи хранятся в хэшированном виде, что предотвращает их компрометацию даже в случае утечки базы данных.

5. Таблица пользовательских разрешений (`user_permissions`) реализует ACL-модель прав доступа, позволяя назначать индивидуальные разрешения конкретным пользователям вне зависимости от их роли.

6. Таблица моделей прав (`permission_models`) содержит описание доступных моделей управления правами: простую (RBAC), иерархическую и ACL.

Ключевые модули и их взаимодействие

Модуль аутентификации отвечает за проверку подлинности пользователей. Он включает регистрацию новых пользователей с валидацией входных данных, процесс входа с проверкой учетных данных и обязательной двухфакторной аутентификацией, а также управление сессиями через JWT-токены.

Модуль авторизации реализует систему контроля доступа на основе ролей. Он проверяет права пользователя на выполнение конкретных операций, используя три различные модели: простую RBAC (на основе ролей), иерархическую (с наследованием прав) и ACL (индивидуальные разрешения).

Модуль управления пользователями предоставляет функциональность для работы с пользовательскими данными: просмотр

профилей, блокировка и разблокировка аккаунтов, смена паролей с обязательной проверкой текущего пароля, обновление контактной информации.

Модуль управления ролями позволяет администраторам создавать новые роли с кастомизированными наборами прав, просматривать существующие роли и статистику их использования.

Модуль двухфакторной аутентификации обеспечивает дополнительный уровень безопасности, требуя подтверждения входа через временный код, отправляемый на email или телефон пользователя. В учебных целях код не отправляется реально, а выводится в консоль сервера.

Система безопасности

Безопасность данных является приоритетом в разработанной системе. Хэширование паролей осуществляется с использованием алгоритма Argon2, который считается современным стандартом и устойчив к атакам перебора. Для каждого пароля генерируется уникальная соль, что исключает возможность использования радужных таблиц.

Управление сессиями построено на основе JWT-токенов с разделением на access-токены (короткое время жизни – 30 минут) и refresh-токены (длительное время жизни – 7 дней). Это ограничивает ущерб в случае компрометации access-токена и позволяет безопасно обновлять сессии без повторного ввода пароля.

Валидация входных данных выполняется на нескольких уровнях: проверка формата email-адресов, валидация телефонных номеров в российском формате, контроль минимальной длины и сложности паролей. Все данные очищаются перед использованием для предотвращения инъекционных атак.

Механизмы контроля доступа

Система реализует четкое разделение прав между тремя базовыми ролями:

Администратор обладает полным доступом ко всем функциям системы: просмотр полной информации о всех пользователях (включая телефоны), блокировка и разблокировка любых аккаунтов, смена ролей пользователей, удаление учетных записей, создание и управление ролями, доступ к системным настройкам.

Модератор имеет ограниченные права управления пользователями: просмотр списка всех пользователей, но только с видимостью email (телефоны скрыты), возможность блокировать и разблокировать только обычных пользователей (не администраторов). Модератор не может изменять роли пользователей или удалять аккаунты.

Обычный пользователь может работать только со своим профилем: просматривать и редактировать свои данные, менять пароль с подтверждением текущего, обновлять контактную информацию. Доступ к информации о других пользователях полностью отсутствует.

API и интеграционные возможности

Система предоставляет полноценный REST API с версионированием эндпоинтов. API документирован с использованием OpenAPI спецификации и доступен через Swagger UI и ReDoc интерфейсы. Каждый эндпоинт включает проверку прав доступа, валидацию входных данных и обработку ошибок с информативными сообщениями.

Консольный клиент демонстрирует все возможности API через удобное текстовое меню. Интерфейс динамически адаптируется к роли пользователя, показывая только доступные функции. Навигация интуитивно понятна, все сообщения представлены на русском языке, что облегчает использование системы.

Процесс работы системы

При первом запуске сервер автоматически инициализирует базу данных: создает необходимые таблицы, добавляет три модели прав, три базовые роли и тестового администратора. Это позволяет сразу начать работу с системой без дополнительной настройки.

Регистрация нового пользователя начинается с ввода уникального имени пользователя, валидного email-адреса (обязательно), телефонного номера в российском формате (опционально) и надежного пароля минимальной длины 8 символов. Система автоматически назначает новому пользователю роль «user» по умолчанию.

Процесс входа включает два этапа: сначала проверяются логин и пароль, затем требуется подтверждение через двухфакторную аутентификацию. Пользователь может выбрать получение кода по email или SMS. После успешной аутентификации создается новая сессия и генерируются JWT-токены.

Работа в системе зависит от роли пользователя. Администраторы видят полное меню со всеми функциями управления системой. Модераторы имеют доступ к ограниченному набору инструментов для управления пользователями. Обычные пользователи работают только со своим профилем, что наглядно демонстрирует принцип минимальных привилегий.

Расширяемость и возможности развития

Модульная архитектура системы позволяет легко добавлять новую функциональность. Поддержка новых моделей прав может быть реализована путем добавления соответствующих классов в модуль авторизации. Интеграция с внешними системами возможна через REST API или добавление новых методов аутентификации.

Инструкция по применению

Разработанная система аутентификации и авторизации запускается и используется по четкому алгоритму, который позволяет быстро ознакомиться со всеми ее возможностями. Процесс начинается с одновременного запуска серверной и клиентской частей в двух отдельных терминалах, что создает полноценную рабочую среду для тестирования.

В первом терминале необходимо перейти в папку сервера и выполнить команду запуска, то есть просто запустить `main.py`. Сервер автоматически инициализирует базу данных, создаст все необходимые таблицы и заполнит их тестовыми данными, включая предустановленного администратора системы. Успешный запуск сопровождается сообщением о готовности сервера на локальном адресе с портом 8000.

Параллельно во втором терминале запускается консольное клиентское приложение `console_client.py` из соответствующей папки. Клиент по умолчанию настроен на подключение к локальному серверу и после запуска отображает главное меню с вариантами действий для неавторизованного пользователя.

Первым практическим шагом рекомендуется войти в систему под учетной записью тестового администратора, которая создается автоматически при инициализации базы данных. После выбора пункта входа в меню клиента пользователь вводит логин и пароль администратора, после чего система запрашивает подтверждение через двухфакторную аутентификацию. Код подтверждения не отправляется по реальным каналам, а отображается в консоли сервера, откуда его нужно скопировать и ввести в клиентском приложении. Этот механизм демонстрирует работу двухфакторной аутентификации 2FA без необходимости настройки реальных сервисов отправки сообщений.

Успешная аутентификация открывает доступ к расширенному меню администратора, где представлен полный набор функций системы.

Пользователь может последовательно изучить каждую возможность: просмотреть список всех зарегистрированных пользователей с полной контактной информацией, ознакомиться с существующими ролями и их описаниями, проверить собственные права доступа к различным ресурсам. Особый интерес представляет функция создания новых ролей с кастомными правами, где можно экспериментировать с различными комбинациями разрешений.

Для демонстрации принципа разделения прав рекомендуется выйти из системы администратора и зарегистрировать нового обычного пользователя. Процесс регистрации включает ввод уникального имени пользователя, валидного email-адреса, телефонного номера в российском формате и пароля минимальной длины. После регистрации система автоматически назначает новому пользователю роль «user» и предлагает войти в аккаунт.

Вход под обычным пользователем наглядно показывает разницу в доступных функциях: меню сокращается до базовых операций с собственным профилем. Пользователь может просматривать и редактировать свои данные, менять пароль с обязательным подтверждением текущего, обновлять контактную информацию, но не имеет доступа к управлению другими пользователями или системным настройкам.

Чтобы продемонстрировать динамическое изменение прав, нужно снова войти в систему под администратором, найти в списке пользователей только что созданного аккаунта и изменить его роль на "moderator". После этого при повторном входе под измененным пользователем меню расширяется: появляется возможность просматривать список всех пользователей и выполнять операции блокировки, но с ограничениями – модератор видит только email-адреса других пользователей, телефонные номера остаются скрытыми, а администраторов нельзя блокировать.

Работа с системой включает практическое тестирование различных сценариев. Например, можно проверить, как разные роли видят одни и те же данные: администратор наблюдает полную информацию о всех пользователях, модератор – ограниченный набор данных, а обычный пользователь не видит других аккаунтов вообще. Система также позволяет тестировать границы прав доступа, пытаясь выполнить действия, не разрешенные текущей ролью, и анализируя соответствующие сообщения об ошибках.

Для более глубокого понимания архитектуры системы рекомендуется использовать встроенную API-документацию, доступную через веб-браузер. Интерактивная документация в формате Swagger позволяет напрямую тестировать все эндпоинты системы, отправлять запросы и анализировать ответы, что особенно полезно для понимания работы бэкенд-части.

В процессе эксплуатации могут возникнуть типичные ситуации, требующие решения. Если клиент не может подключиться к серверу, следует проверить активность серверного процесса и отсутствие блокировки порта фаерволом. При проблемах с двухфакторной аутентификацией нужно убедиться, что код берется из правильной консоли сервера и не истек его срок действия. В случае необходимости тестирования на другом порту можно изменить конфигурацию сервера перед запуском.

Завершение работы с системой должно выполняться корректно: сначала осуществляется выход из аккаунта через соответствующий пункт меню в клиентском приложении, затем сервер останавливается стандартной комбинацией клавиш в его терминале. При последующем перезапуске система сохраняет всех созданных пользователей и внесенные изменения, демонстрируя устойчивость хранения данных.

Таким образом, система предоставляет интуитивно понятный и последовательный процесс освоения, позволяя на практике изучить все аспекты современной аутентификации и авторизации, от базовой регистрации до сложных сценариев управления правами доступа в многопользовательской среде.

Код на языке программирования Python

Листинг 1 – Код консольного интерфейса приложения

`\client\console_client.py`

```
import requests
import json
import sys
from typing import Optional, Dict, Any, List
import time

class AuthClient:
    def __init__(self, base_url: str = "http://localhost:8000"):
        self.base_url = base_url
        self.access_token: Optional[str] = None
        self.refresh_token: Optional[str] = None
        self.current_user: Optional[Dict] = None
        self.user_role: str = ""
        self.user_id: Optional[int] = None

    def _make_request(self, method: str, endpoint: str, **kwargs):
        """Отправка HTTP запроса"""
        url = f"{self.base_url}{endpoint}"

        # Отладочная информация
        print(f"\nЗапрос: {method} {url}")

        headers = kwargs.get('headers', {})
        if self.access_token:
            headers['Authorization'] = f"Bearer {self.access_token}"
            print(f"Используется токен")

        kwargs['headers'] = headers

        try:
            # Добавляем timeout
            if 'timeout' not in kwargs:
                kwargs['timeout'] = 10

            # Логируем тело запроса
            if 'json' in kwargs:
                print(f"Тело запроса: {kwargs['json']}")

            response = requests.request(method, url, **kwargs)

            print(f"Статус: {response.status_code}")

            if response.status_code == 401:
                print("Ошибка авторизации. Возможно, токен истек.")
```



```

        self.access_token = None
        self.current_user = None
        self.user_role = ""
        self.user_id = None

    # Проверяем статус
    if response.status_code >= 400:
        print(f"Ошибка {response.status_code}: {response.text}")

    response.raise_for_status()

    if response.content:
        result = response.json()
        return result
    return {}

except requests.exceptions.HTTPError as e:
    print(f"HTTP ошибка: {e}")
    if hasattr(e.response, 'text'):
        print(f"Ответ сервера: {e.response.text}")
    return {}
except requests.exceptions.RequestException as e:
    print(f"Ошибка соединения: {e}")
    return {}

def clear_screen(self):
    """Очистка экрана (кросс-платформенная)"""
    import os
    os.system('cls' if os.name == 'nt' else 'clear')

def print_header(self):
    """Вывод заголовка"""
    print("\n" + "=" * 60)
    print("СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ".center(60))
    print("=" * 60)

    if self.current_user:
        username = self.current_user.get('username', 'Неизвестно')
        role = self.user_role
        role_display = {
            'admin': 'АДМИНИСТРАТОР',
            'moderator': 'МОДЕРАТОР',
            'user': 'ПОЛЬЗОВАТЕЛЬ'
        }.get(role, role.upper())

        print(f"Текущий пользователь: {username} - {role_display}")
        print("-" * 60)

def register(self):
    """Регистрация нового пользователя"""

```

```

self.clear_screen()
print("\n" + "=" * 60)
print("РЕГИСТРАЦИЯ НОВОГО ПОЛЬЗОВАТЕЛЯ".center(60))
print("=" * 60)

username = input("\nИмя пользователя: ").strip()
email = input("Email (обязательно): ").strip()
phone = input("Телефон (опционально): ").strip()
password = input("Пароль (мин. 8 символов): ").strip()

if len(password) < 8:
    print("\nПароль должен быть не менее 8 символов")
    input("\nНажмите Enter для продолжения...")
    return

data = {
    "username": username,
    "email": email,
    "password": password
}

if phone:
    data["phone"] = phone

print("\nРегистрация...")
result = self._make_request("POST", "/api/v1/register", json=data)

if result and "username" in result:
    print(f"\nПользователь {username} успешно зарегистрирован!")
    print("ВНИМАНИЕ: Для входа требуется двухфакторная аутентификация!")
else:
    print(f"\nОшибка регистрации")

input("\nНажмите Enter для продолжения...")

def login(self):
    """Вход в систему с 2ФА"""
    self.clear_screen()
    print("\n" + "=" * 60)
    print("ВХОД В СИСТЕМУ".center(60))
    print("=" * 60)

    username = input("\nИмя пользователя: ").strip()
    password = input("Пароль: ").strip()

    # Формируем данные для отправки
    data = {
        "username": username,
        "password": password,
        "grant_type": "password",

```

```

    "client_id": "console_client",
    "client_secret": "secret"
}

print(f"\nПроверка учетных данных для пользователя {username}...")

# Используем Form данные как в FastAPI
result = self._make_request("POST", "/api/v1/login", data=data)

# Обработка различных сценариев
if result and result.get("requires_contact_info"):
    print("\nДля входа требуется указать email или номер телефона")
    print("1.Email")
    print("2.SMS")
    choice = input("Выберите способ (1 или 2): ").strip()

    if choice == "1":
        selected_channel = "email"
        contact_info = input("Введите email: ").strip()
    elif choice == "2":
        selected_channel = "sms"
        contact_info = input("Введите номер телефона: ").strip()
    else:
        print("Неверный выбор")
        input("\nНажмите Enter для продолжения...")
        return

    data["selected_channel"] = selected_channel
    data["contact_info"] = contact_info
    result = self._make_request("POST", "/api/v1/login", data=data)

if result and result.get("requires_channel_selection"):
    print("\nВыберите способ получения кода:")
    channels = result.get("available_channels", [])
    for i, channel in enumerate(channels, 1):
        print(f"{i}. {channel.upper()}")

    try:
        choice = int(input("Ваш выбор: ").strip())
        selected_channel = channels[choice - 1]
        data["selected_channel"] = selected_channel
        result = self._make_request("POST", "/api/v1/login", data=data)
    except:
        print("Неверный выбор")
        input("\nНажмите Enter для продолжения...")
        return

if result and result.get("requires_mfa"):
    channel = result.get("selected_channel", "unknown").upper()
    print(f"\nКод отправлен на {channel}")

```

```

print("Проверьте консоль сервера для получения кода!")

code = input("\nВведите 6-значный код: ").strip()
data["mfa_code"] = code
result = self._make_request("POST", "/api/v1/login", data=data)

if result and 'access_token' in result:
    self.access_token = result['access_token']
    self.refresh_token = result.get('refresh_token')
    self.current_user = result.get('user', {})
    self.user_role = self.current_user.get('role', '').lower()
    self.user_id = self.current_user.get('id')

    print(f"\nУСПЕШНЫЙ ВХОД!")
    print(f"Пользователь: {self.current_user.get('username')}")
    print(f"Роль: {self.current_user.get('role')}")

    # Показываем доступные возможности
    self.show_capabilities()
else:
    print("Ошибка входа")

input("\nНажмите Enter для продолжения...")

def show_capabilities(self):
    """Показать доступные возможности для текущей роли"""
    print("\n" + "=" * 60)
    print("ВАШИ ВОЗМОЖНОСТИ:".center(60))
    print("=" * 60)

    if self.user_role == "admin":
        print("\nВАМ ДОСТУПНЫ ВСЕ ФУНКЦИИ АДМИНИСТРАТОРА:")
        print(" 1. Просмотр всех пользователей")
        print(" 2. Блокировка/разблокировка пользователей")
        print(" 3. Смена ролей пользователей")
        print(" 4. Удаление пользователей")
        print(" 5. Создание новых ролей")
        print(" 6. Просмотр всех ролей")

    elif self.user_role == "moderator":
        print("\nВАМ ДОСТУПНЫ ФУНКЦИИ МОДЕРАТОРА:")
        print(" 1. Просмотр всех пользователей")
        print(" 2. Блокировка/разблокировка пользователей")
        print(" НЕ доступно: смена ролей, удаление пользователей")

    elif self.user_role == "user":
        print("\nВАМ ДОСТУПНЫ ФУНКЦИИ ПОЛЬЗОВАТЕЛЯ:")
        print(" 1. Просмотр своего профиля")
        print(" 2. Смена пароля")
        print(" 3. Обновление контактной информации")

```

```

        print(" НЕ доступно: просмотр других пользователей")

def show_profile(self):
    """Информация о пользователе"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)
    print("ИНФОРМАЦИЯ О ПОЛЬЗОВАТЕЛЕ".center(60))
    print("=" * 60)

    result = self._make_request("GET", "/api/v1/me")

    if result:
        print(f"\nИмя пользователя: {result.get('username')}")
        print(f"Email: {result.get('email')} or 'Не указан'")
        print(f"Телефон: {result.get('phone')} or 'Не указан'")
        role_info = result.get('role')
        if isinstance(role_info, dict):
            role_name = role_info.get('name', 'Не назначена')
        else:
            role_name = role_info or 'Не назначена'
        print(f"Роль: {role_name}")
        print(f"2FA: {'Включена' if result.get('mfa_enabled') else 'Выключена'}")
        print(f"Зарегистрирован: {result.get('created_at')}")
        print(f"Статус: {'Активен' if result.get('is_active') else 'Заблокирован'}")

        # Сохраняем ID пользователя
        self.user_id = result.get('id')
    else:
        print("Ошибка получения информации")

    input("\nНажмите Enter для продолжения...")

def list_all_users(self):
    """Показать всех пользователей"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)

```

```

print("СПИСОК ВСЕХ ПОЛЬЗОВАТЕЛЕЙ".center(60))
print("=" * 60)

result = self._make_request("GET", "/api/v1/users")

if result and isinstance(result, list):
    print(f"\nВсего пользователей: {len(result)}")
    print("-" * 60)

    for i, user in enumerate(result, 1):
        status = "Активен" if user.get('is_active') else "Заблокирован"
        role = user.get('role', 'Не назначена')
        print(f"{i}. {user.get('username')} ({role}) - {status}")

        if user.get('email'):
            print(f"    {user.get('email')}")

        if user.get('phone') and self.user_role == 'admin':
            print(f"    {user.get('phone')}")

        print(f"    ID: {user.get('id')}")
        print()

# Предлагаем действия для админа/модератора
if self.user_role in ['admin', 'moderator']:
    print("-" * 60)
    print("ВОЗМОЖНЫЕ ДЕЙСТВИЯ:")

    if self.user_role == 'admin':
        print(" • Введите номер пользователя для управления")
        print(" • 'b' - заблокировать пользователя")
        print(" • 'r' - сменить роль пользователя")
        print(" • 'd' - удалить пользователя")
    elif self.user_role == 'moderator':
        print(" • Введите номер пользователя для управления")
        print(" • 'b' - заблокировать/разблокировать пользователя")

    action = input("\nВыберите действие (или Enter для выхода): ").strip().lower()

    if action.isdigit():
        user_index = int(action) - 1
        if 0 <= user_index < len(result):
            self.manage_user(result[user_index])
    elif action == 'b' and self.user_role in ['admin', 'moderator']:
        self.block_user_interactive()
    elif action == 'r' and self.user_role == 'admin':
        self.change_role_interactive()
    elif action == 'd' and self.user_role == 'admin':
        self.delete_user_interactive()

```

```

elif result and 'detail' in result:
    print(f"\n{result['detail']}")
else:
    print("\nНе удалось получить список пользователей")

input("\nНажмите Enter для продолжения...")

def manage_user(self, user_data: Dict):
    """Управление конкретным пользователем"""
    username = user_data.get('username')
    user_id = user_data.get('id')

    print(f"\nУправление пользователем: {username}")
    print(f"ID: {user_id}, Роль: {user_data.get('role')}")

    if self.user_role == 'admin':
        print("\nДоступные действия:")
        print("1. Блокировать/разблокировать")
        print("2. Сменить роль")
        print("3. Удалить пользователя")

        choice = input("Выберите действие (1-3): ").strip()

        if choice == "1":
            self.toggle_user_status(user_id, username)
        elif choice == "2":
            self.change_user_role(user_id, username)
        elif choice == "3":
            self.delete_user(user_id, username)

    elif self.user_role == 'moderator':
        print("\nДоступные действия:")
        print("1. Блокировать/разблокировать")

        choice = input("Выберите действие (1): ").strip()

        if choice == "1":
            self.toggle_user_status(user_id, username)

def block_user_interactive(self):
    """Интерактивная блокировка пользователя"""
    username = input("\nВведите имя пользователя для блокировки/разблокировки: ").strip()

    # Получаем ID пользователя по username
    users_result = self._make_request("GET", "/api/v1/users")
    if users_result and isinstance(users_result, list):
        for user in users_result:
            if user.get('username') == username:
                self.toggle_user_status(user.get('id'), username)

```

```

        return

    print(f"Пользователь '{username}' не найден")

def toggle_user_status(self, user_id: int, username: str):
    """Блокировка/разблокировка пользователя"""
    if user_id == self.user_id:
        print("Нельзя изменить статус самому себе")
        input("\nНажмите Enter для продолжения...")
        return

    # Сначала получаем текущий статус
    user_info = self._make_request("GET", f"/api/v1/users/{user_id}")

    if not user_info:
        print(f"Не удалось получить информацию о пользователе {username}")
        return

    current_status = user_info.get('is_active', True)
    new_status = not current_status

    action = "разблокировать" if new_status else "заблокировать"

    confirm = input(f"\nВы уверены, что хотите {action} пользователя {username}? (y/n): ").lower()

    if confirm == 'y':
        data = {"is_active": new_status}
        result = self._make_request("PATCH", f"/api/v1/users/{user_id}/status", json=data)

        if result and 'message' in result:
            print(f"\n{result['message']}")
        else:
            print(f"\nОшибка при изменении статуса пользователя")
    else:
        print("Действие отменено")

    input("\nНажмите Enter для продолжения...")

def change_role_interactive(self):
    """Интерактивная смена роли"""
    username = input("\nВведите имя пользователя для смены роли: ").strip()

    # Получаем ID пользователя по username
    users_result = self._make_request("GET", "/api/v1/users")
    if users_result and isinstance(users_result, list):
        for user in users_result:
            if user.get('username') == username:
                self.change_user_role(user.get('id'), username)
                return

```



```

print(f"Пользователь '{username}' не найден")

def change_user_role(self, user_id: int, username: str):
    """Смена роли пользователя"""
    if user_id == self.user_id:
        print("Нельзя изменить роль самому себе")
        input("\nНажмите Enter для продолжения...")
        return

    print("\nДоступные роли:")
    print("1. admin (администратор)")
    print("2. moderator (модератор)")
    print("3. user (обычный пользователь)")

    choice = input("\nВыберите новую роль (1-3): ").strip()

    if choice == "1":
        role_name = "admin"
    elif choice == "2":
        role_name = "moderator"
    elif choice == "3":
        role_name = "user"
    else:
        print("Неверный выбор")
        return

    confirm = input(f"\nНазначить роль '{role_name}' пользователю {username}? (y/n): ")
    confirm = confirm.lower()

    if confirm == 'y':
        data = {"role_name": role_name}
        result = self._make_request("PATCH", f"/api/v1/users/{user_id}/role", json=data)

        if result and 'message' in result:
            print(f"\n{result['message']}")
        else:
            print(f"\nОшибка при смене роли")
        else:
            print("Действие отменено")

    input("\nНажмите Enter для продолжения...")

def delete_user_interactive(self):
    """Интерактивное удаление пользователя"""
    username = input("\nВведите имя пользователя для удаления: ").strip()

    # Получаем ID пользователя по username
    users_result = self._make_request("GET", "/api/v1/users")
    if users_result and isinstance(users_result, list):

```

```

        for user in users_result:
            if user.get('username') == username:
                self.delete_user(user.get('id'), username)
                return

        print(f"Пользователь '{username}' не найден")

def delete_user(self, user_id: int, username: str):
    """Удаление пользователя"""
    if user_id == self.user_id:
        print("Нельзя удалить самого себя")
        input("\nНажмите Enter для продолжения...")
        return

    confirm = input(f"\nВНИМАНИЕ: Вы уверены, что хотите удалить пользователя {username}? (y/n): ").lower()

    if confirm == 'y':
        result = self._make_request("DELETE", f"/api/v1/users/{user_id}")

        if result and 'message' in result:
            print(f"\n{result['message']}")
        else:
            print(f"\nОшибка при удалении пользователя")
        else:
            print("Действие отменено")

    input("\nНажмите Enter для продолжения...")

def change_password(self):
    """Смена пароля"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)
    print("СМЕНА ПАРОЛЯ".center(60))
    print("=" * 60)

    current = input("\nТекущий пароль: ").strip()
    new = input("\nНовый пароль (мин. 8 символов): ").strip()

    if len(new) < 8:
        print("\nНовый пароль должен быть не менее 8 символов")
        input("\nНажмите Enter для продолжения...")
        return

```

```

data = {
    "current_password": current,
    "new_password": new
}

result = self._make_request("POST", "/api/v1/change-password", json=data)

if result and 'message' in result:
    print(f"\n{result['message']}")
else:
    print("\nОшибка смены пароля")

input("\nНажмите Enter для продолжения...")

def update_contact_info(self):
    """Обновление контактной информации"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)
    print("ОБНОВЛЕНИЕ КОНТАКТНОЙ ИНФОРМАЦИИ".center(60))
    print("=" * 60)

    print("\nЧто вы хотите обновить?")
    print("1.Email")
    print("2.Номер телефона")

    choice = input("\nВыберите вариант (1-2): ").strip()

    if choice == "1":
        channel = "email"
        contact_info = input("Новый email: ").strip()
    elif choice == "2":
        channel = "sms"
        contact_info = input("Новый номер телефона (+79991234567): ").strip()
    else:
        print("Неверный выбор")
        input("\nНажмите Enter для продолжения...")
        return

    data = {
        "channel": channel,
        "contact_info": contact_info
    }

```

```

result = self._make_request("POST", "/api/v1/profile/update-contact", json=data)

if result and 'success' in result and result['success']:
    print(f"\n{result['message']}")
else:
    print("\nОшибка обновления контактной информации")

input("\nНажмите Enter для продолжения...")

def view_roles(self):
    """Просмотр всех ролей"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)
    print("СПИСОК ВСЕХ РОЛЕЙ".center(60))
    print("=" * 60)

    result = self._make_request("GET", "/api/v1/roles")

    if result and isinstance(result, list):
        for role in result:
            print(f"\n{role.get('name', 'Неизвестно')}")
            print(f"  Описание: {role.get('description', 'Нет описания')}")
            print(f"  Пользователей: {role.get('user_count', 0)}")
            print(f"  ID: {role.get('id')}")

            if role.get('parent_role'):
                print(f"  Наследует от: {role.get('parent_role')}")
        else:
            print("\nНе удалось получить список ролей")

    input("\nНажмите Enter для продолжения...")

def create_role(self):
    """Создание новой роли"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

```

```

print("\n" + "=" * 60)
print("СОЗДАНИЕ НОВОЙ РОЛИ".center(60))
print("=" * 60)

name = input("\nНазвание роли: ").strip()
description = input("Описание роли: ").strip()

print("\nПримеры разрешений (введите через запятую):")
print("users:view_all, users:block, profile:view, roles:create")
print("Для простоты оставьте пустым для стандартных прав")
permissions_input = input("Разрешения (оставьте пустым для пропуска): ").strip()

permissions = {}
if permissions_input:
    for perm in permissions_input.split(','):
        perm = perm.strip()
        if ':' in perm:
            resource, action = perm.split(':', 1)
            if resource not in permissions:
                permissions[resource] = []
            permissions[resource].append(action)

data = {
    "name": name,
    "description": description,
    "permissions": permissions if permissions else {"profile": ["view", "edit"]}
}

result = self._make_request("POST", "/api/v1/roles", json=data)

if result and 'name' in result:
    print(f"\nРоль '{result['name']}' успешно создана!")
    print(f"  ID: {result.get('id')}")
else:
    print("\nОшибка создания роли")

input("\nНажмите Enter для продолжения...")

def check_permission(self):
    """Проверка прав доступа"""
    self.clear_screen()
    self.print_header()

    if not self.access_token:
        print("\nСначала войдите в систему")
        input("\nНажмите Enter для продолжения...")
        return

    print("\n" + "=" * 60)
    print("ПРОВЕРКА ПРАВ ДОСТУПА".center(60))

```

```

print("=" * 60)

print("\nПримеры ресурсов: users, roles, profile, system")
print("Примеры действий: view_all, block, delete, change_role, view, create, edit")

resource = input("\nРесурс: ").strip()
action = input("Действие: ").strip()

result = self._make_request("GET", f"/api/v1/check-permission/{resource}/{action}")

if result and 'has_permission' in result:
    if result['has_permission']:
        print(f"\nУ вас есть право '{action}' на ресурс '{resource}'")
    else:
        print(f"\nУ вас НЕТ права '{action}' на ресурс '{resource}'")
else:
    print("\nОшибка проверки прав")

input("\nНажмите Enter для продолжения...")

def logout(self):
    """Выход из системы"""
    self.access_token = None
    self.refresh_token = None
    self.current_user = None
    self.user_role = ""
    self.user_id = None
    print("\nВыход выполнен")
    input("\nНажмите Enter для продолжения...")

def get_role_menu(self) -> List[str]:
    """Получить меню для текущей роли"""
    base_menu = [
        "Показать профиль",
        "Сменить пароль",
        "Обновить контакты",
        "Проверить права",
        "Выход из системы",
        "Выход из программы"
    ]

    if self.user_role == "admin":
        return [
            "Просмотр всех пользователей",
            "Просмотр всех ролей",
            "Создать новую роль",
        ] + base_menu

    elif self.user_role == "moderator":
        return [

```

```

        "Просмотр всех пользователей",
    ] + base_menu

else: # user
    return base_menu

def main_menu(self):
    """Главное меню"""
    while True:
        self.clear_screen()
        self.print_header()

        if not self.current_user:
            # Меню для неавторизованного пользователя
            print("\nГЛАВНОЕ МЕНЮ:")
            print("1.Регистрация")
            print("2.Вход в систему")
            print("3.Выход из программы")

            choice = input("\nВыберите действие: ").strip()

            if choice == "1":
                self.register()
            elif choice == "2":
                self.login()
            elif choice == "3":
                print("\nДо свидания!")
                break
            else:
                print("Неверный выбор")
                input("\nНажмите Enter для продолжения...")

        else:
            # Меню для авторизованного пользователя
            menu_items = self.get_role_menu()

            print("\nВАШЕ МЕНЮ:")
            for i, item in enumerate(menu_items, 1):
                print(f"{i}. {item}")

            try:
                choice = int(input("\nВыберите действие: ").strip())

                if 1 <= choice <= len(menu_items):
                    selected = menu_items[choice - 1]

                    if selected == "Показать профиль":
                        self.show_profile()
                    elif selected == "Просмотр всех пользователей":
                        self.list_all_users()

```

```

        elif selected == "Просмотр всех ролей":
            self.view_roles()
        elif selected == "Создать новую роль":
            self.create_role()
        elif selected == "Сменить пароль":
            self.change_password()
        elif selected == "Обновить контакты":
            self.update_contact_info()
        elif selected == "Проверить права":
            self.check_permission()
        elif selected == "Выход из системы":
            self.logout()
        elif selected == "Выход из программы":
            if self.access_token:
                self.logout()
            print("\nДо свидания!")
            break
    else:
        print("Неверный выбор")
        input("\nНажмите Enter для продолжения...")

except ValueError:
    print("Введите номер пункта меню")
    input("\nНажмите Enter для продолжения...")

def main():
    """Основная функция"""
    client = AuthClient()
    client.main_menu()

if __name__ == "__main__":
    main()

```

Листинг 2 – Код основного файла FastAPI-приложения \server\main.py

```

from fastapi import FastAPI, Depends, HTTPException, status, Request, Form
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from datetime import timedelta, datetime
from typing import Optional, Dict, List, Any
import uvicorn
import models
import schemas
import crud
from database import engine, get_db
from security import (
    create_access_token,
    create_refresh_token,

```



```

    verify_token,
    get_password_hash,
    ACCESS_TOKEN_EXPIRE_MINUTES
)
from auth import (
    get_current_user,
    get_current_active_user,
    permission_required,
    PermissionChecker
)
from mfa import MFAManager
from validators import validate_email, validate_phone
from contextlib import asynccontextmanager

```

LIFESPAN МЕНЕДЖЕР

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Управление событиями запуска/остановки"""
    print("Запуск сервера...")
    await init_database()
    yield
    print("Остановка сервера...")

```

```

app = FastAPI(
    title="Система аутентификации и авторизации",
    description="Лабораторная работа №6 - Четкое разделение прав по ролям",
    version="3.0",
    docs_url="/docs",
    redoc_url="/redoc",
    lifespan=lifespan
)

```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

ИНИЦИАЛИЗАЦИЯ БД

```

async def init_database():
    """Создание тестовых данных"""
    print("Инициализация базы данных...")

```

```

# Создаем таблицы
models.Base.metadata.create_all(bind=engine)

```

```

db = next(get_db())
try:
    # Создаем модели прав
    permission_models = [
        {"id": 1, "name": "simple", "description": "Простая модель прав"},
        {"id": 2, "name": "hierarchical", "description": "Иерархическая модель"},
        {"id": 3, "name": "acl", "description": "ACL модель"},
    ]

    for model_data in permission_models:
        if not db.query(models.PermissionModel).filter(models.PermissionModel.id ==
model_data["id"]).first():
            db.add(models.PermissionModel(**model_data))

    db.commit()

    # Создаем роли с ПРАВИЛЬНЫМИ правами
    roles_data = [
        {
            "id": 1,
            "name": "admin",
            "description": "Администратор - полный доступ",
            "permissions": {
                "users": ["view_all", "block", "delete", "change_role", "view_details"],
                "roles": ["view", "create", "edit", "delete"],
                "system": ["manage"],
                "profile": ["view", "edit"],
                "**": ["*"]
            },
            "permission_model_id": 1
        },
        {
            "id": 2,
            "name": "moderator",
            "description": "Модератор - может блокировать пользователей",
            "permissions": {
                "users": ["view_all", "block"], # ВАЖНО: view_all должен быть
                "profile": ["view", "edit"]
            },
            "permission_model_id": 1
        },
        {
            "id": 3,
            "name": "user",
            "description": "Обычный пользователь - только свой профиль",
            "permissions": {
                "profile": ["view", "edit"]
            },
            "permission_model_id": 1
        }
    ]

```

```

    }
]

for role_data in roles_data:
    if not db.query(models.Role).filter(models.Role.name == role_data["name"]).first():
        role = models.Role(
            name=role_data["name"],
            description=role_data["description"],
            permissions=role_data["permissions"],
            permission_model_id=role_data["permission_model_id"]
        )
        db.add(role)

db.commit()

# Создаем главного администратора системы
general_users = [
    {
        "username": "Жук Анастасия Валерьевна",
        "email": "nastyazhuk@mail.ru",
        "password": "nastya521",
        "role_id": 1,
        "phone": "+79999999999"
    }
]

for user_data in general_users:
    if not crud.get_user_by_username(db, user_data["username"]):
        hashed_pw = get_password_hash(user_data["password"])
        user = models.User(
            username=user_data["username"],
            email=user_data["email"],
            phone=user_data["phone"],
            password_hash=hashed_pw,
            role_id=user_data["role_id"],
            mfa_enabled=True,
            is_active=True
        )
        db.add(user)

db.commit()

print("База данных инициализирована!")

except Exception as e:
    print(f"Ошибка инициализации: {e}")
    db.rollback()
finally:
    db.close()

```

РЕГИСТРАЦИЯ

```
@app.post("/api/v1/register", response_model=schemas.UserResponse)
async def register_user(
    user_data: schemas.UserCreate,
    db: Session = Depends(get_db)
):
    """Регистрация нового пользователя"""
    try:
        # Все новые пользователи - обычные user
        user_data.role_id = 3
        user = crud.create_user(db, user_data)
        return user
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Ошибка: {str(e)}")
```

ЛОГИН С 2FA

```
@app.post("/api/v1/login")
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    mfa_code: Optional[str] = Form(None),
    selected_channel: Optional[str] = Form(None),
    contact_info: Optional[str] = Form(None),
    db: Session = Depends(get_db)
):
    """Вход с обязательной 2FA"""

    print(f"Попытка входа: {form_data.username}")

    user = crud.authenticate_user(db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="Неверный логин или пароль")

    if not user.is_active:
        raise HTTPException(status_code=400, detail="Пользователь заблокирован")

    mfa_manager = MFAManager(db)

    # Если нет кода 2FA
    if not mfa_code:
        if selected_channel and contact_info:
            success, message = crud.update_user_contact(db, user, selected_channel,
contact_info)
            if not success:
                raise HTTPException(status_code=400, detail=message)

        channels = mfa_manager.get_available_channels(user)
```

```

if not channels:
    return {
        "requires_contact_info": True,
        "message": "Укажите email или телефон для 2FA"
    }

if len(channels) == 1:
    channel = channels[0]
elif selected_channel and selected_channel in channels:
    channel = selected_channel
else:
    return {
        "requires_mfa": True,
        "requires_channel_selection": True,
        "available_channels": channels,
        "message": "Выберите способ получения кода"
    }

code = mfa_manager.generate_mfa_code()
mfa_manager.save_mfa_code(user, code)
mfa_manager.send_mfa_code_emulation(user, code, channel)

return {
    "requires_mfa": True,
    "selected_channel": channel,
    "message": f"Код отправлен на {channel}"
}

else:
    # Проверка кода 2FA
    success, message = mfa_manager.verify_mfa_code(user, mfa_code)
    if not success:
        raise HTTPException(status_code=401, detail=f"Неверный код: {message}")

    # Создание сессии
    session = crud.create_session(db, user.id)

    # Токены
    access_token = create_access_token(
        data={"sub": user.username, "user_id": user.id}
    )
    refresh_token = create_refresh_token(data={"sub": user.username})

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer",
        "user": {
            "id": user.id,

```

```

        "username": user.username,
        "email": user.email,
        "role": user.role.name if user.role else "user",
        "is_active": user.is_active
    }
}

```

ПРОФИЛЬ ПОЛЬЗОВАТЕЛЯ

```
@app.get("/api/v1/me")
```

```
async def get_my_profile(current_user: models.User = Depends(get_current_active_user)):
```

```
    """Мой профиль"""
```

```
    return {
```

```
        "id": current_user.id,
```

```
        "username": current_user.username,
```

```
        "email": current_user.email,
```

```
        "phone": current_user.phone,
```

```
        "role": current_user.role.name if current_user.role else None,
```

```
        "role_id": current_user.role_id,
```

```
        "is_active": current_user.is_active,
```

```
        "mfa_enabled": current_user.mfa_enabled,
```

```
        "created_at": current_user.created_at
```

```
    }
```

```
@app.post("/api/v1/change-password")
```

```
async def change_my_password(
```

```
    password_data: dict,
```

```
    current_user: models.User = Depends(get_current_active_user),
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """Смена своего пароля"""
```

```
    current_password = password_data.get("current_password")
```

```
    new_password = password_data.get("new_password")
```

```
    if not current_password or not new_password:
```

```
        raise HTTPException(status_code=400, detail="Требуется текущий и новый пароль")
```

```
    if len(new_password) < 8:
```

```
        raise HTTPException(status_code=400, detail="Новый пароль должен быть не менее 8 символов")
```

```
    success = crud.change_user_password(db, current_user, current_password, new_password)
```

```
    if not success:
```

```
        raise HTTPException(status_code=400, detail="Неверный текущий пароль")
```

```
    return {"message": "Пароль изменен успешно"}
```

```
@app.post("/api/v1/profile/update-contact")
```

```
async def update_my_contact(
```

```

        contact_data: dict,
        current_user: models.User = Depends(get_current_active_user),
        db: Session = Depends(get_db)
    ):
        """Обновление своих контактов"""
        channel = contact_data.get("channel")
        contact_info = contact_data.get("contact_info")

        if channel not in ["email", "sms"]:
            raise HTTPException(status_code=400, detail="Канал должен быть 'email' или 'sms'")

        if not contact_info:
            raise HTTPException(status_code=400, detail="Контактная информация обязательна")

        success, message = crud.update_user_contact(db, current_user, channel, contact_info)
        if not success:
            raise HTTPException(status_code=400, detail=message)
        return {"success": True, "message": "Контактная информация обновлена успешно"}

```

УНИФИЦИРОВАННЫЕ ЭНДПОИНТЫ

```

@app.get("/api/v1/users")
async def get_all_users(
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Получить список всех пользователей (для админа и модератора)"""

    # Простая проверка по ролям
    if not current_user.role:
        raise HTTPException(status_code=403, detail="У вас нет роли")

    # Админ и модератор могут видеть пользователей
    if current_user.role.name not in ["admin", "moderator"]:
        raise HTTPException(
            status_code=403,
            detail="Недостаточно прав для просмотра списка пользователей"
        )

    users = db.query(models.User).all()

    # Формируем ответ в зависимости от роли
    result = []
    for user in users:
        user_data = {
            "id": user.id,
            "username": user.username,
            "role": user.role.name if user.role else None,

```

```

        "is_active": user.is_active,
        "created_at": user.created_at
    }

    # Админ видит всю информацию
    if current_user.role.name == "admin":
        user_data["email"] = user.email
        user_data["phone"] = user.phone
        user_data["role_id"] = user.role_id
    # Модератор видит только email
    elif current_user.role.name == "moderator":
        user_data["email"] = user.email

    result.append(user_data)

return result

@app.get("/api/v1/users/{user_id}")
async def get_user_by_id(
    user_id: int,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Получить информацию о пользователе по ID"""
    user = crud.get_user(db, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="Пользователь не найден")

    # Можно смотреть только свой профиль, если не админ/модератор
    if not current_user.role:
        raise HTTPException(status_code=403, detail="У вас нет роли")

    if user_id != current_user.id and current_user.role.name not in ["admin", "moderator"]:
        raise HTTPException(
            status_code=403,
            detail="Можно просматривать только свой профиль"
        )

    # Формируем ответ
    response = {
        "id": user.id,
        "username": user.username,
        "role": user.role.name if user.role else None,
        "is_active": user.is_active,
        "created_at": user.created_at
    }

    # Проверяем, что показывать
    if user_id == current_user.id:

```



```

        # Свой профиль - все данные
        response["email"] = user.email
        response["phone"] = user.phone
        response["role_id"] = user.role_id
        response["mfa_enabled"] = user.mfa_enabled
    elif current_user.role.name == "admin":
        # Админ видит все
        response["email"] = user.email
        response["phone"] = user.phone
        response["role_id"] = user.role_id
        response["mfa_enabled"] = user.mfa_enabled
    elif current_user.role.name == "moderator":
        # Модератор видит только email
        response["email"] = user.email

    return response

@app.patch("/api/v1/users/{user_id}/status")
async def update_user_status(
    user_id: int,
    status_data: dict,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Блокировка/разблокировка пользователя (для админа и модератора)"""
    if user_id == current_user.id:
        raise HTTPException(status_code=400, detail="Нельзя изменить статус самому себе")

    user = crud.get_user(db, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="Пользователь не найден")

    # Проверка роли
    if not current_user.role:
        raise HTTPException(status_code=403, detail="У вас нет роли")

    if current_user.role.name not in ["admin", "moderator"]:
        raise HTTPException(
            status_code=403,
            detail="Недостаточно прав для блокировки пользователей"
        )

    # Модератор не может блокировать администраторов
    if current_user.role.name == "moderator":
        if user.role and user.role.name == "admin":
            raise HTTPException(
                status_code=403,
                detail="Модератор не может блокировать администраторов"
            )

```

```

    )

    new_status = status_data.get("is_active")
    if new_status is None:
        raise HTTPException(status_code=400, detail="Поле 'is_active' обязательно")

    user.is_active = bool(new_status)
    db.commit()

    action = "разблокирован" if user.is_active else "заблокирован"
    return {"message": f"Пользователь {user.username} успешно {action}"}

@app.patch("/api/v1/users/{user_id}/role")
async def change_user_role(
    user_id: int,
    role_data: dict,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Смена роли пользователя (только для админа)"""
    if user_id == current_user.id:
        raise HTTPException(status_code=400, detail="Нельзя изменить роль самому себе")

    user = crud.get_user(db, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="Пользователь не найден")

    # Проверка роли (только админ)
    if not current_user.role or current_user.role.name != "admin":
        raise HTTPException(
            status_code=403,
            detail="Только администраторы могут менять роли пользователей"
        )

    role_name = role_data.get("role_name")
    if not role_name:
        raise HTTPException(status_code=400, detail="Поле 'role_name' обязательно")

    role = crud.get_role_by_name(db, role_name)
    if not role:
        raise HTTPException(status_code=404, detail=f"Роль '{role_name}' не найдена")

    user.role_id = role.id
    db.commit()

    return {"message": f"Роль пользователя {user.username} успешно изменена на {role_name}"}

```

```

@app.delete("/api/v1/users/{user_id}")
async def delete_user(
    user_id: int,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Удаление пользователя (только для админа)"""
    if user_id == current_user.id:
        raise HTTPException(status_code=400, detail="Нельзя удалить самого себя")

    user = crud.get_user(db, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="Пользователь не найден")

    # Проверка роли (только админ)
    if not current_user.role or current_user.role.name != "admin":
        raise HTTPException(
            status_code=403,
            detail="Только администраторы могут удалять пользователей"
        )

    db.delete(user)
    db.commit()

    return {"message": f"Пользователь {user.username} успешно удален"}

```

РОЛИ

```

@app.get("/api/v1/roles")
async def get_all_roles(
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Получить список всех ролей (для админа)"""
    # Проверка роли (только админ)
    if not current_user.role or current_user.role.name != "admin":
        raise HTTPException(
            status_code=403,
            detail="Только администраторы могут просматривать роли"
        )

    roles = db.query(models.Role).all()

    result = []
    for role in roles:
        role_data = {
            "id": role.id,
            "name": role.name,
            "description": role.description,

```

```

        "permission_model_id": role.permission_model_id,
        "user_count": len(role.users) if role.users else 0
    }

    if role.parent_role:
        role_data["parent_role"] = role.parent_role.name

    result.append(role_data)

return result

@app.post("/api/v1/roles")
async def create_new_role(
    role_data: dict,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Создание новой роли (только для админа)"""
    # Проверка роли (только админ)
    if not current_user.role or current_user.role.name != "admin":
        raise HTTPException(
            status_code=403,
            detail="Только администраторы могут создавать роли"
        )

    name = role_data.get("name")
    description = role_data.get("description", "")
    permissions = role_data.get("permissions", {})

    if not name:
        raise HTTPException(status_code=400, detail="Поле 'name' обязательно")

    # Проверяем, существует ли уже роль с таким именем
    existing_role = crud.get_role_by_name(db, name)
    if existing_role:
        raise HTTPException(status_code=400, detail=f"Роль с именем '{name}' уже существует")

    # Создаем новую роль
    role = models.Role(
        name=name,
        description=description,
        permissions=permissions,
        permission_model_id=1 # Простая модель по умолчанию
    )

    db.add(role)
    db.commit()
    db.refresh(role)

```

```

return {
    "id": role.id,
    "name": role.name,
    "description": role.description,
    "permissions": role.permissions
}

```

ПРОВЕРКА ПРАВ

```

@app.get("/api/v1/check-permission/{resource}/{action}")
async def check_permission(
    resource: str,
    action: str,
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Проверка, есть ли у пользователя право"""
    checker = PermissionChecker(db)
    has_perm = checker.check_permission(current_user, resource, action)

```

```

return {
    "has_permission": has_perm,
    "user": current_user.username,
    "role": current_user.role.name if current_user.role else None,
    "resource": resource,
    "action": action
}

```

```

@app.get("/api/v1/my-permissions")
async def get_my_permissions(
    current_user: models.User = Depends(get_current_active_user),
    db: Session = Depends(get_db)
):
    """Получение всех моих прав"""
    checker = PermissionChecker(db)

    # Проверяем основные права
    permissions = {
        "users": {
            "view_all": checker.check_permission(current_user, "users", "view_all"),
            "block": checker.check_permission(current_user, "users", "block"),
            "delete": checker.check_permission(current_user, "users", "delete"),
            "change_role": checker.check_permission(current_user, "users", "change_role")
        },
        "roles": {
            "view": checker.check_permission(current_user, "roles", "view"),
            "create": checker.check_permission(current_user, "roles", "create"),

```

```

        "edit": checker.check_permission(current_user, "roles", "edit"),
        "delete": checker.check_permission(current_user, "roles", "delete")
    },
    "profile": {
        "view": checker.check_permission(current_user, "profile", "view"),
        "edit": checker.check_permission(current_user, "profile", "edit")
    }
}

return {
    "user": current_user.username,
    "role": current_user.role.name if current_user.role else None,
    "permissions": permissions
}

```

ОБЩАЯ ИНФОРМАЦИЯ

```

@app.get("/")
async def root():
    """Главная страница"""
    return {
        "message": "Система аутентификации с четким разделением прав",
        "version": "1.0",
        "docs": "/docs",
        "roles": {
            "admin": "Полный доступ ко всем функциям",
            "moderator": "Может просматривать пользователей и блокировать их",
            "user": "Только свой профиль"
        },
        "general_users": [
            {"username": "Жук Анастасия Валерьевна", "password": "nastya521", "role":
"admin"}],
    ]
}

```

ЗАПУСК

```

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

Листинг 3 – Код определения SQLAlchemy-моделей \server\models.py

```

from sqlalchemy import Column, Integer, String, Boolean, DateTime, ForeignKey, JSON,
Text
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from database import Base

```

```

class PermissionModel(Base):
    __tablename__ = "permission_models"

```

```

id = Column(Integer, primary_key=True, index=True)
name = Column(String, unique=True, index=True, nullable=False)
description = Column(String)
is_active = Column(Boolean, default=True)

```

```

roles = relationship("Role", back_populates="permission_model")

```

```

class Role(Base):

```

```

    __tablename__ = "roles"

```

```

id = Column(Integer, primary_key=True, index=True)
name = Column(String, unique=True, index=True, nullable=False)
description = Column(String)
permissions = Column(JSON, default=dict)
parent_role_id = Column(Integer, ForeignKey("roles.id"), nullable=True)
permission_model_id = Column(Integer, ForeignKey("permission_models.id"), default=1)

```

```

parent_role = relationship("Role", remote_side=[id], backref="child_roles")
permission_model = relationship("PermissionModel", back_populates="roles")
users = relationship("User", back_populates="role")

```

```

class User(Base):

```

```

    __tablename__ = "users"

```

```

id = Column(Integer, primary_key=True, index=True)
username = Column(String, unique=True, index=True, nullable=False)
email = Column(String, unique=True, index=True)
phone = Column(String)
password_hash = Column(String, nullable=False)
role_id = Column(Integer, ForeignKey("roles.id"), default=3) # По умолчанию user

```

```

mfa_enabled = Column(Boolean, default=True)
mfa_secret = Column(String)
mfa_secret_expires = Column(DateTime)

```

```

is_active = Column(Boolean, default=True)
created_at = Column(DateTime(timezone=True), server_default=func.now())

```

```

role = relationship("Role", back_populates="users")
sessions = relationship("Session", back_populates="user")
api_keys = relationship("ApiKey", back_populates="user")
acl_permissions = relationship("UserPermission", back_populates="user")

```

```

class Session(Base):

```

```

    __tablename__ = "sessions"

```

```

id = Column(Integer, primary_key=True, index=True)
user_id = Column(Integer, ForeignKey("users.id"))
session_token = Column(String, unique=True, index=True)
refresh_token = Column(String, unique=True, index=True)
expires_at = Column(DateTime(timezone=True))
created_at = Column(DateTime(timezone=True), server_default=func.now())
user_agent = Column(String)
ip_address = Column(String)

user = relationship("User", back_populates="sessions")

```

```

class ApiKey(Base):
    __tablename__ = "api_keys"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    key = Column(String, unique=True, index=True, nullable=False)
    name = Column(String, nullable=False)
    scopes = Column(JSON, default=list)
    expires_at = Column(DateTime(timezone=True))
    is_active = Column(Boolean, default=True)
    last_used_at = Column(DateTime(timezone=True))
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    user = relationship("User", back_populates="api_keys")

```

```

class UserPermission(Base):
    __tablename__ = "user_permissions"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    resource = Column(String, nullable=False)
    action = Column(String, nullable=False)
    granted = Column(Boolean, default=True)

    user = relationship("User", back_populates="acl_permissions")

```

Листинг 4 – Код Pydantic-схем для валидации входных/выходных данных \server\schemas.py

```

from pydantic import BaseModel, EmailStr, Field, validator
from typing import Optional, List, Dict, Any
from datetime import datetime

# User schemas
class UserBase(BaseModel):
    username: str

```



```
email: Optional[EmailStr] = None
phone: Optional[str] = None
```

```
class UserCreate(UserBase):
    password: str = Field(..., min_length=8)
    role_id: Optional[int] = 2
```

```
class UserLogin(BaseModel):
    username: str
    password: str
```

```
class UserChangePassword(BaseModel):
    current_password: str
    new_password: str = Field(..., min_length=8)
```

```
class UserResponse(UserBase):
    id: int
    is_active: bool
    role_id: int
    mfa_enabled: bool
    created_at: datetime
```

```
class Config:
    from_attributes = True
```

```
# Role schemas
```

```
class RoleBase(BaseModel):
    name: str
    description: Optional[str] = None
    permissions: Dict[str, List[str]] = {}
```

```
class RoleCreate(RoleBase):
    parent_role_id: Optional[int] = None
    permission_model_id: Optional[int] = 1
```

```
class RoleResponse(RoleBase):
    id: int
```

```
class Config:
    from_attributes = True
```

```
# Token schemas
```

```

class Token(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"

class TokenData(BaseModel):
    username: Optional[str] = None
    user_id: Optional[int] = None
    scope: Optional[str] = ""

# Permission schemas
class PermissionModelCreate(BaseModel):
    name: str
    description: Optional[str] = None

class UserPermissionCreate(BaseModel):
    resource: str
    action: str
    granted: bool = True

# API Key schemas
class ApiKeyCreate(BaseModel):
    name: str
    scopes: List[str] = []
    expires_in_days: Optional[int] = 90

class ApiKeyResponse(BaseModel):
    id: int
    name: str
    scopes: List[str]
    expires_at: Optional[datetime]
    is_active: bool
    last_used_at: Optional[datetime]
    created_at: datetime

class Config:
    from_attributes = True

```

Листинг 5 – Код модуля авторизации с проверкой прав доступа и зависимостями FastAPI \server\auth.py

```

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
from typing import List, Optional, Tuple

```

```

import models
from database import get_db
from security import verify_token

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/login")

class PermissionChecker:
    def __init__(self, db: Session):
        self.db = db

    def check_permission_simple(self, user: models.User, resource: str, action: str) -> bool:
        # Если пользователь админ - всегда разрешаем
        if user.role and user.role.name == "admin":
            return True

        if not user.role or not user.role.permissions:
            return False

        permissions = user.role.permissions

        # Проверка через * (полный доступ)
        if "*" in permissions:
            if permissions["*"] == ["*"]:
                return True
            elif action in permissions["*"]:
                return True

        # Проверка конкретного ресурса
        if resource in permissions:
            # Проверяем конкретное действие
            if action in permissions[resource]:
                return True
            # Проверяем доступ через * для ресурса
            if "*" in permissions[resource]:
                return True

        return False

    def check_permission_hierarchical(self, user: models.User, resource: str, action: str) ->
bool:
        def check_role_permissions(role: models.Role, resource: str, action: str) -> bool:
            if role.permissions and resource in role.permissions:
                if action in role.permissions[resource]:
                    return True

            if role.parent_role:
                return check_role_permissions(role.parent_role, resource, action)

        return False

```

```

    if not user.role:
        return False

    return check_role_permissions(user.role, resource, action)

def check_permission_acl(self, user: models.User, resource: str, action: str) -> bool:
    acl_permission = self.db.query(models.UserPermission).filter(
        models.UserPermission.user_id == user.id,
        models.UserPermission.resource == resource,
        models.UserPermission.action == action
    ).first()

    if acl_permission:
        return acl_permission.granted

    return self.check_permission_simple(user, resource, action)

def check_permission(self, user: models.User, resource: str, action: str) -> bool:
    if not user.role:
        return False

    # Админ всегда имеет доступ
    if user.role.name == "admin":
        return True

    # Для модераторов проверяем права специально
    if user.role.name == "moderator":
        if resource == "users" and action == "view_all":
            return True
        if resource == "users" and action == "block":
            return True
        if resource == "profile" and action in ["view", "edit"]:
            return True

    # Используем простую модель по умолчанию
    return self.check_permission_simple(user, resource, action)

# Dependencies
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Неверные учетные данные",
        headers={"WWW-Authenticate": "Bearer"},
    )

```

```

payload = verify_token(token)
if payload is None:
    raise credentials_exception

username: str = payload.get("sub")
token_type: str = payload.get("type")

if username is None or token_type != "access":
    raise credentials_exception

user = db.query(models.User).filter(models.User.username == username).first()
if user is None:
    raise credentials_exception

if not user.is_active:
    raise HTTPException(status_code=400, detail="Пользователь неактивен")

return user

async def get_current_active_user(
    current_user: models.User = Depends(get_current_user)
):
    if not current_user.is_active:
        raise HTTPException(status_code=400, detail="Пользователь неактивен")
    return current_user

def permission_required(resource: str, action: str):
    def permission_checker(
        current_user: models.User = Depends(get_current_active_user),
        db: Session = Depends(get_db)
    ):
        checker = PermissionChecker(db)
        if not checker.check_permission(current_user, resource, action):
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Недостаточно прав"
            )
        return current_user

    return permission_checker

```

Листинг 6 – Код альтернативных методов аутентификации

\server\auth_methods.py

```

from fastapi import HTTPException, status, Depends, Request
from fastapi.security import HTTPBasic, HTTPBasicCredentials, HTTPBearer,
OAuth2PasswordBearer, APIKeyHeader, \
APIKeyQuery

```

```

from sqlalchemy.orm import Session
from typing import Optional, Tuple
import secrets
import base64
import hashlib
from datetime import datetime

from database import get_db
import models
import crud
from security import verify_password, verify_token

# Security schemes
basic_auth = HTTPBasic()
bearer_auth = HTTPBearer()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/oauth/token")
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
api_key_query = APIKeyQuery(name="api_key", auto_error=False)

class AuthenticationMethods:
    def __init__(self, db: Session):
        self.db = db

    def authenticate_basic(self, username: str, password: str) -> Optional[models.User]:
        user = crud.get_user_by_username(self.db, username)
        if not user:
            return None

        if not verify_password(password, user.password_hash):
            return None

        return user

    def authenticate_api_key(self, api_key: str) -> Optional[models.User]:
        # Хэшируем ключ для поиска в БД
        key_hash = hashlib.sha256(api_key.encode()).hexdigest()

        key_record = self.db.query(models.ApiKey).filter(
            models.ApiKey.key == key_hash,
            models.ApiKey.is_active == True
        ).first()

        if not key_record:
            return None

        if key_record.expires_at and key_record.expires_at < datetime.utcnow():
            key_record.is_active = False
            self.db.commit()
            return None

```

```

key_record.last_used_at = datetime.utcnow()
self.db.commit()

return key_record.user

def authenticate_session(self, session_token: str) -> Optional[models.User]:
    session = self.db.query(models.Session).filter(
        models.Session.session_token == session_token,
        models.Session.expires_at > datetime.utcnow()
    ).first()

    if not session:
        return None

    return session.user

# Dependencies для разных методов аутентификации
async def get_current_user_basic(
    credentials: HTTPBasicCredentials = Depends(basic_auth),
    db: Session = Depends(get_db)
) -> models.User:
    auth_methods = AuthenticationMethods(db)
    user = auth_methods.authenticate_basic(credentials.username, credentials.password)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неверные учетные данные Basic Auth",
            headers={"WWW-Authenticate": "Basic"},
        )

    if not user.is_active:
        raise HTTPException(status_code=400, detail="Пользователь неактивен")

    return user

async def get_current_user_api_key(
    api_key_header: Optional[str] = Depends(api_key_header),
    api_key_query: Optional[str] = Depends(api_key_query),
    db: Session = Depends(get_db)
) -> models.User:
    auth_methods = AuthenticationMethods(db)

    api_key = api_key_header or api_key_query
    if not api_key:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,

```

```

        detail="API ключ не предоставлен"
    )

    user = auth_methods.authenticate_api_key(api_key)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неверный или неактивный API ключ"
        )

    if not user.is_active:
        raise HTTPException(status_code=400, detail="Пользователь неактивен")

    return user

```

Листинг 7 – Код функций для работы с БД \server\crud.py

```

from sqlalchemy.orm import Session
from sqlalchemy import and_
import models
import schemas
from security import get_password_hash, verify_password
from datetime import datetime, timedelta
import uuid
from validators import validate_email, validate_phone
from typing import Tuple, Optional

# User CRUD
def get_user(db: Session, user_id: int):
    return db.query(models.User).filter(models.User.id == user_id).first()

def get_user_by_username(db: Session, username: str):
    return db.query(models.User).filter(models.User.username == username).first()

def get_user_by_email(db: Session, email: str):
    return db.query(models.User).filter(models.User.email == email).first()

def get_user_by_phone(db: Session, phone: str):
    return db.query(models.User).filter(models.User.phone == phone).first()

def create_user(db: Session, user: schemas.UserCreate) -> models.User:
    if user.email:
        is_valid, message = validate_email(user.email)
        if not is_valid:
            raise ValueError(f"Invalid email: {message}")

```



```

if user.phone:
    is_valid, message = validate_phone(user.phone)
    if not is_valid:
        raise ValueError(f"Invalid phone: {message}")
    user.phone = message

hashed_password = get_password_hash(user.password)
db_user = models.User(
    username=user.username,
    email=user.email,
    phone=user.phone,
    password_hash=hashed_password,
    role_id=user.role_id
)

db.add(db_user)
db.commit()
db.refresh(db_user)
return db_user

def authenticate_user(db: Session, username: str, password: str):
    user = get_user_by_username(db, username)
    if not user:
        return None
    if not verify_password(password, user.password_hash):
        return None
    return user

def update_user_contact(db: Session, user: models.User, channel: str, contact_info: str) ->
Tuple[bool, str]:
    if channel == 'email':
        is_valid, message = validate_email(contact_info)
        if not is_valid:
            return False, message

        existing = get_user_by_email(db, contact_info)
        if existing and existing.id != user.id:
            return False, "Email уже используется"

        user.email = contact_info

    elif channel == 'sms':
        is_valid, message = validate_phone(contact_info)
        if not is_valid:
            return False, message

        existing = get_user_by_phone(db, contact_info)

```

```

        if existing and existing.id != user.id:
            return False, "Номер телефона уже используется"

        user.phone = contact_info

    else:
        return False, "Неизвестный канал"

    db.commit()
    return True, "Контактная информация обновлена"

def change_user_password(db: Session, user: models.User, current_password: str,
new_password: str) -> bool:
    if not verify_password(current_password, user.password_hash):
        return False

    user.password_hash = get_password_hash(new_password)
    db.commit()
    return True

# Role CRUD
def get_role(db: Session, role_id: int):
    return db.query(models.Role).filter(models.Role.id == role_id).first()

def get_role_by_name(db: Session, name: str):
    return db.query(models.Role).filter(models.Role.name == name).first()

def create_role(db: Session, role: schemas.RoleCreate):
    db_role = models.Role(
        name=role.name,
        description=role.description,
        permissions=role.permissions,
        parent_role_id=role.parent_role_id,
        permission_model_id=role.permission_model_id
    )
    db.add(db_role)
    db.commit()
    db.refresh(db_role)
    return db_role

def assign_role_to_user(db: Session, user: models.User, role_name: str):
    role = get_role_by_name(db, role_name)
    if role and user.role_id != role.id:
        user.role_id = role.id
        db.commit()

```

```
return user
```

```
# Session CRUD
```

```
def create_session(db: Session, user_id: int, user_agent: str = None, ip_address: str = None):
```

```
    session_token = str(uuid.uuid4())
```

```
    refresh_token = str(uuid.uuid4())
```

```
    expires_at = datetime.utcnow() + timedelta(days=7)
```

```
    db_session = models.Session(  
        user_id=user_id,  
        session_token=session_token,  
        refresh_token=refresh_token,  
        expires_at=expires_at,  
        user_agent=user_agent,  
        ip_address=ip_address  
    )
```

```
    db.add(db_session)
```

```
    db.commit()
```

```
    db.refresh(db_session)
```

```
    return db_session
```

```
def get_session(db: Session, session_token: str):
```

```
    return db.query(models.Session).filter(  
        models.Session.session_token == session_token,  
        models.Session.expires_at > datetime.utcnow()  
    ).first()
```

```
def delete_session(db: Session, session_token: str):
```

```
    session = db.query(models.Session).filter(models.Session.session_token ==  
    session_token).first()
```

```
    if session:
```

```
        db.delete(session)
```

```
        db.commit()
```

```
        return True
```

```
    return False
```

```
# API Key CRUD
```

```
def get_api_key(db: Session, api_key_hash: str):
```

```
    return db.query(models.ApiKey).filter(  
        models.ApiKey.key == api_key_hash,  
        models.ApiKey.is_active == True  
    ).first()
```

Листинг 8 – Код конфигурации подключения к PostgreSQL и создания сессий SQLAlchemy \server\database.py

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql://postgres:nasya_521!@localhost:5433/db_timp")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Листинг 9 – Код модуля двухфакторной аутентификации \server\mfa.py

```
import random
import string
from datetime import datetime, timedelta
from sqlalchemy.orm import Session
import models
from typing import Tuple, List, Optional

class MFAManager:
    def __init__(self, db: Session):
        self.db = db

    def generate_mfa_code(self, length: int = 6) -> str:
        return "".join(random.choices(string.digits, k=length))

    def save_mfa_code(self, user: models.User, code: str) -> None:
        user.mfa_secret = code
        user.mfa_secret_expires = datetime.utcnow() + timedelta(minutes=10)
        self.db.commit()

    def verify_mfa_code(self, user: models.User, code: str) -> Tuple[bool, str]:
        if not user.mfa_secret:
            return False, "Код не был запрошен"
```

```

if user.mfa_secret_expires and user.mfa_secret_expires < datetime.utcnow():
    return False, "Код истек"

if user.mfa_secret != code:
    return False, "Неверный код"

user.mfa_secret = None
user.mfa_secret_expires = None
self.db.commit()

return True, "Код подтвержден"

def get_available_channels(self, user: models.User) -> List[str]:
    channels = []
    if user.email:
        channels.append('email')
    if user.phone:
        channels.append('sms')
    return channels

def send_mfa_code_emulation(self, user: models.User, code: str, channel: str) -> dict:
    if channel == 'email':
        print(f"\n[ЭМУЛЯЦИЯ] Email отправлен на {user.email}")
        print(f"  Код подтверждения: {code}")
        print(f"  Тема: Ваш код для входа в систему")
        print(f"  Сообщение: Используйте этот код для завершения входа: {code}\n")

    elif channel == 'sms':
        print(f"\n[ЭМУЛЯЦИЯ] SMS отправлено на {user.phone}")
        print(f"  Код подтверждения: {code}")
        print(f"  Сообщение: Ваш код для входа: {code}\n")

    return {
        "success": True,
        "channel": channel,
        "message": f"Код отправлен на {channel}",
        "test_code": code
    }

```

Листинг 10 – Код криптографических функций \server\security.py

```

from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from typing import Optional
import os
from dotenv import load_dotenv

load_dotenv()

```

```

# Конфигурация
SECRET_KEY = os.getenv("SECRET_KEY", "fb649g4bg8nj8gdf8g7c1f5g23aw")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
REFRESH_TOKEN_EXPIRE_DAYS = 7

# Argon2 контекст
pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

    to_encode.update({"exp": expire, "type": "access"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode.update({"exp": expire, "type": "refresh"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str) -> Optional[dict]:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        return None

```

Листинг 11 – Код валидации email и телефонных номеров\server\validators.py

```

import re
from typing import Tuple, Optional

```

```

def validate_email(email: str) -> Tuple[bool, str]:
    if not email:
        return False, "Email не может быть пустым"

    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if not re.match(pattern, email):
        return False, "Неверный формат email"

    return True, "OK"

def validate_phone(phone: str) -> Tuple[bool, str]:
    if not phone:
        return False, "Номер телефона не может быть пустым"

    cleaned = re.sub(r'\D', "", phone)

    if len(cleaned) == 11:
        if cleaned.startswith('7') or cleaned.startswith('8'):
            formatted = f"+7{cleaned[1:4]}-{cleaned[4:7]}-{cleaned[7:9]}-{cleaned[9:]}"
            return True, formatted

    elif len(cleaned) == 10 and cleaned.startswith('9'):
        formatted = f"+7{cleaned[0:3]}-{cleaned[3:6]}-{cleaned[6:8]}-{cleaned[8:]}"
        return True, formatted

    return False, "Неверный формат российского номера телефона. Пример:
+79991234567"

```

Примеры работы программы

```
Microsoft Windows [Version 10.0.26100.7462]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.
Active code page: 65001

C:\Users\Анастасия>cd C:\py.project\first\timp_lab6\server

C:\py.project\first\timp_lab6\server>python main.py
INFO: Will watch for changes in these directories: ['C:\\py.project\\first\\timp_lab6\\server']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15140] using WatchFiles
INFO: Started server process [22692]
INFO: Waiting for application startup.
Запуск сервера...
Инициализация базы данных...
База данных инициализирована!
INFO: Application startup complete.
```

Рисунок 1 – Запуск сервера

```
=====
                        СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====

ГЛАВНОЕ МЕНЮ:
1.Регистрация
2.Вход в систему
3.Выход из программы

Выберите действие:
```

Рисунок 2 – Запущенный консольный интерфейс

```
=====
                        РЕГИСТРАЦИЯ НОВОГО ПОЛЬЗОВАТЕЛЯ
=====

Имя пользователя: Калинина Марина Ивановна
Email (обязательно): mary_kalinina_98@gmail.com
Телефон (опционально): +79551625648
Пароль (мин. 8 символов): kalinina

Регистрация...

Запрос: POST http://localhost:8000/api/v1/register
Тело запроса: {'username': 'Калинина Марина Ивановна', 'email': 'mary_kalinina_98@gmail.com', 'password': 'kalinina', 'phone': '+79551625648'}
Статус: 200

Пользователь Калинина Марина Ивановна успешно зарегистрирован!
ВНИМАНИЕ: Для входа требуется двухфакторная аутентификация!

Нажмите Enter для продолжения...
```

Рисунок 3 – Регистрация пользователя (всегда как «user»)


```
Microsoft Windows [Version 10.0.26100.7462]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.
Active code page: 65001

C:\Users\Анастасия>cd C:\py.project\first\timp_lab6\server

C:\py.project\first\timp_lab6\server>python main.py
INFO: Will watch for changes in these directories: ['C:\\py.project\\fir
st\\timp_lab6\\server']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15140] using WatchFiles
INFO: Started server process [22692]
INFO: Waiting for application startup.
Запуск сервера...
Инициализация базы данных...
База данных инициализирована!
INFO: Application startup complete.
INFO: 127.0.0.1:62577 - "POST /api/v1/register HTTP/1.1" 200 OK
Попытка входа: Жук Анастасия Валерьевна
INFO: 127.0.0.1:25146 - "POST /api/v1/login HTTP/1.1" 200 OK
Попытка входа: Жук Анастасия Валерьевна

[ЭМУЛЯЦИЯ] Email отправлен на nasyazhuk@mail.ru
Код подтверждения: 188268
Тема: Ваш код для входа в систему
Сообщение: Используйте этот код для завершения входа: 188268

INFO: 127.0.0.1:64037 - "POST /api/v1/login HTTP/1.1" 200 OK
Попытка входа: Жук Анастасия Валерьевна
INFO: 127.0.0.1:64039 - "POST /api/v1/login HTTP/1.1" 200 OK
|

=====
ВХОД В СИСТЕМУ
=====

Имя пользователя: Жук Анастасия Валерьевна
Пароль: nasya521

Проверка учетных данных для пользователя Жук Анастасия Валерьевна...

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

Выберите способ получения кода:
1. EMAIL
2. SMS
Ваш выбор: 1

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

Код отправлен на EMAIL
Проверьте консоль сервера для получения кода!

Введите 6-значный код: 188268

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

УСПЕШНЫЙ ВХОД!
Пользователь: Жук Анастасия Валерьевна
Роль: admin
```

Рисунок 4 – Вход в систему под администратором

```
=====
СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Жук Анастасия Валерьевна – АДМИНИСТРАТОР
=====

ВАШЕ МЕНЮ:
1. Просмотр всех пользователей
2. Просмотр всех ролей
3. Создать новую роль
4. Показать профиль
5. Сменить пароль
6. Обновить контакты
7. Проверить права
8. Выход из системы
9. Выход из программы

Выберите действие:
```

Рисунок 5 – Возможности администратора

```
=====
Текущий пользователь: Жук Анастасия Валерьевна – АДМИНИСТРАТОР
=====

=====
СПИСОК ВСЕХ ПОЛЬЗОВАТЕЛЕЙ
=====

Запрос: GET http://localhost:8000/api/v1/users
Используется токен
Статус: 200

Всего пользователей: 4
-----
1. Иванов Иван Иванович (moderator) – Активен
   ivan@gmail.com
   89632651548
   ID: 2

2. Петров Петр Петрович (user) – Активен
   petr123@mail.ru
   ID: 3

3. Калинина Марина Ивановна (user) – Активен
   mary_kalinina_98@gmail.com
   +7955-162-56-48
   ID: 4

4. Жук Анастасия Валерьевна (admin) – Активен
   nastyazhuk@mail.ru
   +79999999999
   ID: 1
-----

ВОЗМОЖНЫЕ ДЕЙСТВИЯ:
• Введите номер пользователя для управления
• 'b' – заблокировать пользователя
• 'r' – сменить роль пользователя
• 'd' – удалить пользователя

Выберите действие (или Enter для выхода): |
```

Рисунок 6 – Все пользователи в системе

```

=====
ВОЗМОЖНЫЕ ДЕЙСТВИЯ:
• Введите номер пользователя для управления
• 'b' - заблокировать пользователя
• 'r' - сменить роль пользователя
• 'd' - удалить пользователя

Выберите действие (или Enter для выхода): b

Введите имя пользователя для блокировки/разблокировки: Калинина Марина Ивановна

Запрос: GET http://localhost:8000/api/v1/users
Используется токен
Статус: 200

Запрос: GET http://localhost:8000/api/v1/users/4
Используется токен
Статус: 200

Вы уверены, что хотите заблокировать пользователя Калинина Марина Ивановна? (y/n): y

Запрос: PATCH http://localhost:8000/api/v1/users/4/status
Используется токен
Тело запроса: {'is_active': False}
Статус: 200

Пользователь Калинина Марина Ивановна успешно заблокирован

Нажмите Enter для продолжения...|

```

Рисунок 7 – Блокировка пользователя

```

=====
Текущий пользователь: Жук Анастасия Валерьевна - АДМИНИСТРАТОР
=====

=====
СПИСОК ВСЕХ ПОЛЬЗОВАТЕЛЕЙ
=====

Запрос: GET http://localhost:8000/api/v1/users
Используется токен
Статус: 200

Всего пользователей: 4
-----
1. Иванов Иван Иванович (moderator) - Активен
   ivan@gmail.com
   89632651548
   ID: 2

2. Петров Петр Петрович (user) - Активен
   petr123@mail.ru
   ID: 3

3. Жук Анастасия Валерьевна (admin) - Активен
   nastyazhuk@mail.ru
   +79999999999
   ID: 1

4. Калинина Марина Ивановна (user) - Заблокирован
   mary_kalinina_98@gmail.com
   +7955-162-56-48
   ID: 4
=====

```

Рисунок 8 – Проверка блокировки пользователя

```

=====
СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Жук Анастасия Валерьевна – АДМИНИСТРАТОР
=====

=====
ИНФОРМАЦИЯ О ПОЛЬЗОВАТЕЛЕ
=====

Запрос: GET http://localhost:8000/api/v1/me
Используется токен
Статус: 200

Имя пользователя: Жук Анастасия Валерьевна
Email: nastyazhuk@mail.ru
Телефон: +79999999999
Роль: admin
2FA: Включена
Зарегистрирован: 2025-12-23T01:35:51.830419
Статус: Активен

Нажмите Enter для продолжения...

```

Рисунок 9 – Информация об администраторе

```

=====
СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Жук Анастасия Валерьевна – АДМИНИСТРАТОР
=====

=====
СМЕНА ПАРОЛЯ
=====

Текущий пароль: nasty521
Новый пароль (мин. 8 символов): nastyadmin

Запрос: POST http://localhost:8000/api/v1/change-password
Используется токен
Тело запроса: {'current_password': 'nasty521', 'new_password': 'nastyadmin'}
Статус: 200

Пароль изменен успешно

Нажмите Enter для продолжения...

```

Рисунок 10 – Смена пароля

```
=====
                               ВХОД В СИСТЕМУ
=====

Имя пользователя: Иванов Иван Иванович
Пароль: 23456789

Проверка учетных данных для пользователя Иванов Иван Иванович...

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

Выберите способ получения кода:
1. EMAIL
2. SMS
Ваш выбор: 1

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

Код отправлен на EMAIL
Проверьте консоль сервера для получения кода!

Введите 6-значный код: 349619

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

УСПЕШНЫЙ ВХОД!
Пользователь: Иванов Иван Иванович
Роль: moderator

=====
                               ВАШИ ВОЗМОЖНОСТИ:
=====

ВАМ ДОСТУПНЫ ФУНКЦИИ МОДЕРАТОРА:
1. Просмотр всех пользователей
2. Блокировка/разблокировка пользователей
НЕ доступно: смена ролей, удаление пользователей
```

Рисунок 11 – Вход в систему модератора

```
=====
СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Иванов Иван Иванович – МОДЕРАТОР
-----

ВАШЕ МЕНЮ:
1. Просмотр всех пользователей
2. Показать профиль
3. Сменить пароль
4. Обновить контакты
5. Проверить права
6. Выход из системы
7. Выход из программы

Выберите действие: |
```

Рисунок 12 – Возможности модератора

```
=====
СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Иванов Иван Иванович – МОДЕРАТОР
-----

=====
СПИСОК ВСЕХ ПОЛЬЗОВАТЕЛЕЙ
=====

Запрос: GET http://localhost:8000/api/v1/users
Используется токен
Статус: 200

Всего пользователей: 4
-----
1. Петров Петр Петрович (user) – Активен
   petr123@mail.ru
   ID: 3
2. Калинина Марина Ивановна (user) – Заблокирован
   mary_kalinina_98@gmail.com
   ID: 4
3. Жук Анастасия Валерьевна (admin) – Активен
   pastyazhuk@mail.ru
   ID: 1
4. Иванов Иван Иванович (moderator) – Активен
   ivan@gmail.com
   ID: 2
-----

ВОЗМОЖНЫЕ ДЕЙСТВИЯ:
• Введите номер пользователя для управления
• 'b' – блокировать/разблокировать пользователя
```

Рисунок 13 – Просмотр пользователей через модератора

```

=====
                          ВХОД В СИСТЕМУ
=====

Имя пользователя: Петров Петр Петрович
Пароль: 12345678

Проверка учетных данных для пользователя Петров Петр Петрович..

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

Код отправлен на EMAIL
Проверьте консоль сервера для получения кода!

Введите 6-значный код: 214961

Запрос: POST http://localhost:8000/api/v1/login
Статус: 200

УСПЕШНЫЙ ВХОД!
Пользователь: Петров Петр Петрович
Роль: user

=====
                          ВАШИ ВОЗМОЖНОСТИ:
=====

ВАМ ДОСТУПНЫ ФУНКЦИИ ПОЛЬЗОВАТЕЛЯ:
1. Просмотр своего профиля
2. Смена пароля
3. Обновление контактной информации
НЕ доступно: просмотр других пользователей

Нажмите Enter для продолжения...|

```

Рисунок 14 – Вход в систему под обычным пользователем

```

=====
                          СИСТЕМА АУТЕНТИФИКАЦИИ И АВТОРИЗАЦИИ
=====
Текущий пользователь: Петров Петр Петрович – ПОЛЬЗОВАТЕЛЬ
-----

ВАШЕ МЕНЮ:
1. Показать профиль
2. Сменить пароль
3. Обновить контакты
4. Проверить права
5. Выход из системы
6. Выход из программы

Выберите действие: |

```

Рисунок 15 – Возможности обычного пользователя

```
=====
                          ВХОД В СИСТЕМУ
=====

Имя пользователя: Калинина Марина Ивановна
Пароль: kalina

Проверка учетных данных для пользователя Калинина Марина Ивановна...

Запрос: POST http://localhost:8000/api/v1/login
Статус: 400
Ошибка 400: {"detail":"Пользователь заблокирован"}
HTTP ошибка: 400 Client Error: Bad Request for url: http://localhost:8000/api/v1/login
Ответ сервера: {"detail":"Пользователь заблокирован"}
Ошибка входа

Нажмите Enter для продолжения...|
```

Рисунок 16 – Попытка входа в систему заблокированного пользователя

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно разработана и реализована полнофункциональная система аутентификации и авторизации, соответствующая современным стандартам информационной безопасности. В процессе работы были изучены и применены ключевые принципы проверки подлинности пользователей, управления сессиями, хэширования паролей и контроля доступа на основе ролей.

Была построена модульная архитектура, включающая серверную часть на FastAPI с REST API и консольное клиентское приложение на Python. Серверная часть обеспечивает всю бизнес-логику, работу с базой данных PostgreSQL и безопасность, в то время как клиентская часть предоставляет удобный интерфейс для взаимодействия пользователей с системой. Реализована многоуровневая система безопасности: пароли хэшируются с использованием стойкого алгоритма Argon2 с уникальной солью, аутентификация проходит с обязательной двухфакторной проверкой, управление сессиями осуществляется через JWT-токены с разделением на access- и refresh-токены.

Особое внимание было уделено системе авторизации и четкому разграничению прав. Внедрены три модели контроля доступа: простая RBAC, иерархическая модель и ACL, что позволяет гибко управлять разрешениями. Система предусматривает три основные роли: администратор (полный доступ ко всем функциям), модератор (ограниченные права управления пользователями) и обычный пользователь (работа только со своим профилем). Такое разделение обеспечивает безопасность и предотвращает несанкционированные действия.

Все модули системы – регистрация, аутентификация с 2FA, управление пользователями и ролями, смена пароля, обновление контактов – были протестированы и работают корректно. Система демонстрирует

отказоустойчивость, валидацию входных данных и информативные сообщения об ошибках. Дополнительно были реализованы такие функции, как проверка прав доступа, создание пользовательских ролей и эмуляция отправки кодов 2FA, что расширяет возможности системы и ее применимость в учебных и практических целях.

Таким образом, в рамках лабораторной работы была создана безопасная, масштабируемая и функциональная система аутентификации и авторизации, которая может служить основой для разработки реальных многопользовательских приложений.